SubWatch PRD v2: Household Optimizer and What-If Analysis

Oct 28, 2026 · @Naman

Which ideas made the cut

Three of the six ideas go into v2, one is folded in as a small add-on, and two are deferred. The rule: keep what turns SubWatch from a set of independent findings into a database-driven household-wide decision, and what also lowers the risks the v1 PRD already names.

#	Idea	Decision	Why
1	What-If Subscription Optimizer	**Core feature**	Combines duplicates, family plans, free spots and bundles (FR-7 to FR-10) into one cheapest-valid-configuration problem. Also settles the v1 open question about alternative suggestions shown as separate findings.
4	Subscription Change Impact Analysis	**Core feature**	Reuses the existing preview-without-changing-data mechanism (FR-17). Answers "what if X changes" with recursive SQL and before/after queries. Shares one engine with idea 1.
3	Confidence-Aware Waste Detection	**Core feature, rule-based**	Targets the two top v1 risks: bad catalog data and people not logging usage. Gives a measurable before/after test.
2	Entitlement Graph with Explanations	**Folded in**	v1 already traverses plan inclusions at any depth. v2 adds only effective dates on inclusions and a "why" path in every finding. No separate graph database.

#	Idea	Decision	Why
5	Fair Cost Sharing Policies	Deferred	Extends FR-19 and is easy to build, but is a separate story from optimization. Candidate for the release after v2.
6	Billing Anomaly Detection	Deferred	Duplicate-charge and price-change checks alone are a modest contribution. Revisit once price history exists.

Ideas 1 and 4 are built as one engine, so the effort is closer to one large feature than two.

Overview and what is new in v2

SubWatch v2 moves from separate per-subscription findings to one household-wide decision: the cheapest set of subscriptions that still gives every member the services they actually need, with a preview of every consequence before anything changes.

One-line pitch: SubWatch finds the cheapest way for a household to keep everything it uses, and shows what any change would do before it happens.

v1 (the DBMS prototype on PostgreSQL, Node/Express and a web dashboard) is unchanged and stays the base. v2 adds three things on top of it:

1. **Household Optimizer.** Searches combinations of plans, bundles and shared tiers under coverage, capacity and household rules, and returns the lowest-cost valid configuration.
2. **What-If Simulator.** Lets users ask "what if we cancel Netflix", "what if our phone plan drops Prime" or "what if a flatmate moves out", and returns affected members, lost coverage, cost change and a cheapest repair.
3. **Confidence scoring.** Every finding carries a High, Medium or Low confidence based on stored evidence, so low-evidence findings ask for verification instead of recommending cancellation.

Novelty claim

Existing trackers flag individual subscriptions that look redundant. SubWatch v2 models entitlements, member coverage and plan capacity together and searches for a cheaper configuration that keeps the required services, then simulates changes against that

model before applying them.

This is a proposed contribution, not a verified claim that no product does it. The evaluation plan includes a short comparison with existing tools and the v1 rule-based optimizer so the claim can be defended.

Problem and goals added by v2

The v1 problems still apply (hidden duplicates, cheaper shared plans, unused subscriptions, untracked charges). v2 adds three gaps that v1 leaves open.

- **Findings are independent and can conflict.** One finding says "switch to a bundle" while another says "drop the least-used app". Applying both can leave a member without a service or save less than expected. The v1 open questions already ask whether these should be shown as a choice.
- **Users can't ask "what if".** v1 previews a fix, but a household can't test a change that comes from outside, such as a phone plan losing a benefit, a price rise or a flatmate leaving.
- **Every finding looks equally trustworthy.** A subscription flagged as unused because the member forgot to log usage looks the same as one confirmed to be included in another plan.

New goals

1. Return one lowest-cost valid configuration for the household, with a rupee saving and a reason for each change.
2. Let users simulate a change and see affected members, services that lose coverage, cost change and a cheapest repair, without modifying data.
3. Show a confidence level and the evidence behind every finding, and keep low-confidence findings out of the savings total.
4. Apply a multi-step recommendation as one all-or-nothing transaction.
5. Measure the new optimizer against the v1 rule-based one on the same data.

Not in v2

- Machine-learning models. Confidence is rule-based.
- Live pricing from providers. Catalog data stays manually maintained.
- Fair-share policy comparison and billing anomaly detection (deferred).
- Login and row-level security stay in the "Next" phase, unchanged from v1.

User stories added by v2

Stories 1 to 8 from v1 remain. New stories:

9. As a household owner, I want one button that shows the cheapest way to keep every service we use, so I don't compare plans by hand.
10. As a member, I want to know that a recommended change still leaves me with every service I use, so I never lose access by mistake.
11. As an owner, I want to ask "what if our phone plan drops Prime" and see who loses what and what it would cost to replace, so I can decide before the change happens.
12. As flatmates, I want to test "what if one of us moves out" and see the new split and whether the shared plan still makes sense.
13. As anyone, I want each suggestion to say how sure the system is and why, so I know when to verify before cancelling.
14. As an owner, I want the system to ask me to check my usage logs when they look incomplete, instead of telling me to cancel things.

Household	Demo household	New scenario to demonstrate
Family of 4 with a family phone plan	Sharma Household	Optimizer collapses duplicate Prime Video and several YouTube plans in one pass. Simulate the phone plan dropping Prime.
Hostel flatmates	Hostel Room A412	Optimizer picks one Family plan over three Individual plans. Simulate one flatmate leaving.
Couple	Iyer Residence	Optimizer prefers a Duo plan where cheaper. Low-confidence finding when usage logs are stale.

Functional requirements

FR-1 to FR-26 are built and unchanged. FR-27 (login) stays planned. The new requirements start at FR-28 and are all planned.

Existing, by area (v1)

Area	IDs	Summary
Households	FR-1, FR-2	Households, service and plan catalog
Subscriptions	FR-3 to FR-6	Add, share, log usage, pause, cancel
Optimizer	FR-7 to FR-16	Seven finding types, explanations, dismissal, auto re-run
Fixes	FR-17, FR-18	Preview and atomic apply
Money	FR-19 to FR-22	Fair split, net transfers, renewals, efficiency score
Import	FR-23 to FR-26	CSV statement import and labeling
Access	FR-27	Login (planned)

New in v2

ID	Area	Requirement	Status
FR-28	Needs	Record which services each member needs. Default: any service the member used in the last 30 days. The owner can add or remove needs by hand.	Planned
FR-29	Optimizer	Household optimizer: given the needs, return the lowest-cost configuration of plans, bundles and shared tiers that covers every member-service need.	Planned
FR-30	Optimizer	Reject any configuration that exceeds plan capacity, shares across households, or leaves a member-service need uncovered. Coverage includes services reached through included plans at any depth.	Planned
FR-31	Optimizer	Show the result as a list of steps (cancel, switch, merge, add member) with the monthly cost before, after and saved, and a reason per step.	Planned
FR-32	Optimizer	Resolve conflicting v1 findings: when findings are alternatives for the same subscriptions, show the optimizer's choice and list the alternatives it rejected with their cost.	Planned

ID	Area	Requirement	Status
FR-33	Fixes	Preview and apply a whole optimizer result using the FR-17 and FR-18 mechanism: one transaction, rolled back entirely if any constraint fails.	Planned
FR-34	What-if	Simulate a change without writing to real tables. Supported changes: cancel a subscription, switch plan, remove a member, remove a plan inclusion (for example a phone plan drops Prime), raise a plan price.	Planned
FR-35	What-if	For each simulation, show members who lose access, services that lose coverage, change in monthly spend, and the cheapest repair found by the optimizer.	Planned
FR-36	What-if	Save a named scenario and re-open it later. Compare two scenarios side by side.	Planned
FR-37	History	Keep effective-dated history of plan prices and plan inclusions, so a past or future date can be evaluated.	Planned
FR-38	Explain	Every finding and optimizer step shows its full reason path (for example Airtel to Amazon Prime to Prime Video), the members covered, and the evidence used.	Planned
FR-39	Confidence	Store evidence rows per finding and compute a confidence level (High, Medium, Low) from them with documented weights.	Planned
FR-40	Confidence	Detect incomplete usage data: if a member has logged nothing for 45 days across all services, usage-based findings for that member are marked Low and ask for verification.	Planned
FR-41	Confidence	Only High and Medium findings count toward the savings total. Low findings appear under "worth a look".	Planned
FR-42	Evaluation	Benchmark mode: generate synthetic households of chosen size and time the optimizer against the v1 rule-based one.	Planned

How the household optimizer works

The optimizer is a constrained cost-minimisation problem, a capacity-limited variant of weighted set cover. SQL supplies the data and checks the rules; the search runs in the Node layer. Households are small (under about 15 members and 30 plans), so an exact search is practical.

1. **Collect needs.** Query the member-service needs (FR-28). Each need is a pair such as (Riya, YouTube Premium).
2. **Expand coverage.** A recursive CTE turns every candidate plan into the set of services it gives, following included plans at any depth. This is the same traversal FR-7 uses.
3. **Build candidates.** Candidates are the household's current plans plus catalog plans of the same services: individual, duo, family and bundles. Monthly price is normalised from the billing cycle.
4. **Search.** Branch and bound over candidate plans, with each member assigned to at most one plan per service. Pruning rules: stop a branch when its cost already exceeds the best valid answer, and when a need can no longer be covered. A greedy answer seeds the upper bound.
5. **Check constraints.** A configuration is valid only if every need is covered, no plan exceeds its member limit, and every sharer belongs to the household. The same rules already live in the database as CHECK constraints and triggers, so an invalid result also fails at write time.
6. **Explain.** Diff the winning configuration against the current one. Each difference becomes a step with its reason path and monthly saving.
7. **Preview and apply.** The steps run inside a transaction that is rolled back for preview (FR-17) or committed for apply (FR-18), under the existing per-household lock.

What-if reuses the same engine. A simulation starts a transaction, applies the hypothetical change (for example deleting a plan inclusion), re-runs steps 1 to 6, records the result, then rolls back. Nothing real changes. The "cheapest repair" in FR-35 is step 4 run on the changed data.

Worked example. A household pays ₹299 for Prime, ₹179 for standalone Prime Video, ₹149 each for two YouTube Individual plans, and ₹119 for Spotify. Prime already covers Prime Video, so the ₹179 plan is dropped. Two Individual plans cost ₹298 and a Family plan costs ₹299, so the optimizer keeps the two Individual plans: a family plan is only recommended

when it is actually cheaper. With a third YouTube member the Family plan wins. Spotify stays because nothing else covers it. The exact figures come from the demo catalog, not from this document.

Data model additions

v2 adds eight tables and two changes to existing ones. Names are proposals; the v1 tables (households, users, services, plans, plan inclusions, subscriptions, members, usage logs, findings, imported transactions) are kept.

Table	Purpose	Key columns and rules
member_needs	A service a member needs (FR-28)	member_id, service_id, source (auto or manual), unique per pair
plan_price_history	Effective-dated plan prices (FR-37)	plan_id, price, valid_from, valid_to; CHECK price >= 0; no overlapping ranges per plan
plan_inclusion_history	Effective-dated inclusions, for example Airtel includes Prime from one date to another	parent_plan_id, included_plan_id, valid_from, valid_to
optimization_runs	One row per optimizer or simulation run	run_id, household_id, kind (optimize or what-if), scenario_name, cost_before, cost_after, created_at
optimization_steps	The steps of a run, with reasons	run_id, step_no, action, subscription_id, plan_id, saving, reason_path
scenarios	Named what-if scenarios (FR-36)	scenario_id, household_id, name, change_type, change_params (JSON)
finding_evidence	Evidence behind each finding (FR-39)	finding_id, evidence_type, source_ref, value, weight
usage_data_quality	Per-member completeness check (FR-40)	member_id, last_logged_at, days_silent, flagged

Changes to existing tables

- `findings` gains `confidence` (High, Medium, Low), `run_id` and `counts_toward_savings`, with a CHECK on the allowed confidence values.
- `plans` price and inclusion reads move to the history tables through a view, so v1 queries keep working.

Integrity rules that stay in the database

- The sum of steps in a committed run must reconcile to cost_before minus cost_after.
- A plan's price ranges and inclusion ranges may not overlap.
- Simulations write only inside a transaction that is rolled back, and the application role has no direct write access to real tables from the simulation path.

DBMS concepts demonstrated

The contribution is meant to be defended as a database project, so each concept below maps to a concrete place in v2 that can be shown live.

Concept	Where in v2	What to show
Normalised schema	Eight new tables, history tables with effective dates	ER diagram and the reason each table exists
Recursive SQL	Coverage expansion through included plans (optimizer step 2) and reason paths (FR-38)	The recursive CTE and the Airtel to Prime to Prime Video path
Constraints and triggers	Price and date-range rules, confidence values, capacity and household checks	An invalid configuration rejected by the database, not just the app
Transactions and atomicity	Apply a multi-step result as one commit (FR-33)	Force a failure in step 3 of 5 and show nothing changed
Isolation and locking	What-if inside a rolled-back transaction, under the per-household lock	Two users simulating and applying at once with no errors or leaks
Temporal data	Price and inclusion history, evaluating a past or future date	Same household costed before and after a price change

Concept	Where in v2	What to show
Aggregation and window functions	Evidence aggregation into confidence, usage silence detection	The query that turns evidence rows into High, Medium or Low
Indexing and query performance	Indexes on coverage and evidence lookups	EXPLAIN ANALYZE before and after, with timings (FR-42)

Non-functional requirements added by v2

v1 rules (integrity in the database, one transaction per change, per-household lock, preview under 0.5 s on demo data, privacy, phone-width dashboard) all still apply to the new features.

Area	Requirement	How v2 will meet it
Performance	Optimizer and what-if return in under 2 s for a household of up to 15 members and 30 candidate plans. Preview of an applied result stays under 0.5 s.	Greedy upper bound, pruning, indexes on coverage lookups. Measured with FR-42 and reported, not assumed.
Correctness	The optimizer never returns a configuration that leaves a need uncovered or breaks a plan limit.	Constraints checked in the search and again by database triggers at write time. Tested against a brute-force search on small households.
Isolation	A simulation never changes real data and never blocks other households.	Runs in a rolled-back transaction, takes only its own household lock.
Determinism	The same data gives the same recommendation.	Ties broken by fewest changes, then lowest plan id.
Explainability	Every step and finding shows its reason path and evidence.	FR-38 and FR-39.
Fallback	If the exact search exceeds the time limit, return the best valid answer found so far and say it may not be the cheapest.	Time-boxed search with the greedy result as the floor.

Evaluation plan and success metrics

On v1 demo data the Sharma Household shows ₹611 a month of savings found on ₹3,037 of spend. That is the baseline v2 is measured against. The targets below are proposals to agree on, not results.

Experiment	Method	Proposed target
Savings vs v1 rules	Run both optimizers on all three demo households and on 50 generated households	v2 saves at least as much as v1 in every case, and strictly more in at least 30% of generated households
Correctness	Compare the optimizer with brute-force search on households small enough to enumerate	Same cost in 100% of cases; zero invalid configurations
Scaling	Time the optimizer at 5, 10, 15, 20 members and 10, 20, 30, 40 plans	Under 2 s up to 15 members and 30 plans; curve reported beyond that
Confidence quality	Hand-label 40 findings on synthetic data as correct or wrong, then compare v1 rules with confidence-aware rules	Fewer wrong cancel recommendations with confidence on, and no more than 10% of correct findings pushed to Low
What-if accuracy	Apply each simulated change for real in a test copy and compare predicted and actual cost	Exact match in 100% of cases
Atomicity	Inject a failure partway through applying a result, 100 times	Zero partial applies

Product metrics (extend the v1 list)

Metric	Proposed target	What it tells us
Optimizer results applied, as a share of results shown	30% or more	The recommendation is worth acting on
What-if runs per household per month	2 or more	Users find the simulator useful

Metric	Proposed target	What it tells us
Wrong cancel recommendations at High confidence	Under 5%	Confidence levels can be trusted
Savings applied per household per month	₹200 or more (unchanged from v1)	The product pays for the time it takes

Release plan

No dates are fixed. The phases show order and what must be true before moving on. Build order inside the DBMS prototype is chosen so each step can be demonstrated on its own.

Phase	Contents	Done when
Built: v1 prototype	7 finding types, preview and apply, bank import, fair split	FR-1 to FR-26 working (already true)
v2a: Foundations	member_needs, history tables, coverage view, reason paths (FR-28, FR-37, FR-38)	Coverage and reason path match v1 findings on all three demo households
v2b: Optimizer	Search, constraint checks, steps, conflict resolution, atomic apply (FR-29 to FR-33)	Matches brute force on small households and beats or equals v1 savings on demo data
v2c: What-if	Simulation in a rolled-back transaction, scenarios, comparison, repair (FR-34 to FR-36)	Predicted cost equals real cost after applying, in tests
v2d: Confidence and benchmark	Evidence, confidence, data-quality check, benchmark mode (FR-39 to FR-42)	Evaluation table filled in with measured numbers
Next	Login and row-level security (FR-27), price history alerts, promo price overrides	Unchanged from v1

Phase	Contents	Done when
Later	Fair-share policy comparison, billing anomaly detection, nightly refresh, usage pulled from apps, mobile app	After the above

If time is short, v2a and v2b alone still deliver the core novelty. v2c reuses the same engine and is the next most valuable. v2d is the measurable-results part and is worth keeping at least in its rule-based minimum form.

Risks, assumptions and open questions

The v1 biggest risk is unchanged: if a plan's price or inclusions are wrong, the optimizer is confidently wrong. v2 raises the stakes because it recommends several changes at once, so the confidence feature and the preview are essential, not optional.

Assumptions

- Plan prices and inclusions are sample data kept up to date by hand.
- A family plan or bundle can be shared by everyone in the household (unchanged from v1).
- A member's needs default to what they used in the last 30 days, which depends on usage being logged.
- Household size stays small enough for an exact search.

Risk	Effect	Mitigation
Wrong catalog price or inclusion	Optimizer recommends a worse configuration	Reason path and evidence on every step, preview before apply, effective-dated history so errors can be traced
Needs inferred from incomplete usage	A member loses a service they use	Needs can be set by hand, data-quality check lowers confidence, optimizer never drops a declared need

Risk	Effect	Mitigation
Search too slow on larger inputs	Poor demo or timeouts	Greedy floor, time limit, indexed coverage, benchmark to find the real limit
Novelty is questioned	Contribution seen as just another tracker	Written novelty claim, comparison with v1 and with two existing trackers, measured results
Simulation leaks into real data	Corrupted household data	Simulations always in a rolled-back transaction, tested by failure injection

Open questions

- Who may apply an optimizer result that cancels someone else's subscription: any member, or only the payer? (carried over from v1)
- Who keeps the plan, inclusion and price catalog current?
- Is a member's need declared once per service, or per plan tier (for example, does a person need Premium or is the free tier fine)?
- Should the optimizer ever suggest a service that no member currently has, such as a bundle that adds a new service for a small extra cost? Default in this draft: no.
- Which two existing subscription trackers will be compared for the novelty claim?